

# 16 — Model Strategy

**Objetivo.** Documentar qué modelo corre detrás de cada rol del swarm y por qué: el sistema de cuatro *tiers* (`premium` / `balanced` / `cheap` / `code`), el agnosticismo de proveedor (cinco providers, dos protocolos, BYO), el reparto entre modelos locales en el pod A40 y MoE gigantes vía API, la lectura del leaderboard agéntico de junio 2026, el descarte explícito de los modelos *abliterated/uncensored*, los *tradeoffs* costo/calidad, y una configuración recomendada lista para copiar. Todo verificado contra el código real en `web/lib/openai.ts` (`MODELS`, `PROVIDERS`, `providerModels`, `resolve`) y `web/lib/roster.ts`.

## 1. The Tier System

Puglit no le pone el mismo cerebro a cada agente. La premisa es de sentido común y está cableada en `web/lib/openai.ts`: **no todo rol necesita el mismo modelo**. Un crítico que extrae nombres de tablas de un blueprint no requiere el mismo poder de razonamiento que el arquitecto que diseña ese blueprint desde una línea de idea. Forzar el modelo más caro en todos lados es quemar plata; forzar el más barato es entregar basura. La respuesta es un sistema de **cuatro tiers**, cada uno con un propósito declarado y un default por proveedor.

El objeto `MODELS` es la única fuente de verdad. Cada tier resuelve a un nombre de modelo, con override por variable de entorno:

```
export const MODELS = {
  premium: process.env.PUGLIT_MODEL_PREMIUM || DEFAULTS.premium, // arquitectura, blueprint,
  balanced: process.env.PUGLIT_MODEL_BALANCED || DEFAULTS.balanced, // specs, critics, seeds
  cheap: process.env.PUGLIT_MODEL_CHEAP || DEFAULTS.cheap, // extracción, normalización
  code: process.env.PUGLIT_MODEL_CODE || DEFAULTS.code, // routes, pages, shell, SQL, etc.
} as const
```

Los cuatro tiers, con la semántica exacta que el comentario del código les asigna:

| Tier                  | Propósito (comentario en <code>openai.ts</code> )                                              | Carga típica               | Sensibilidad a calidad |
|-----------------------|------------------------------------------------------------------------------------------------|----------------------------|------------------------|
| <code>premium</code>  | Architecture, blueprint, discovery, review — <i>reasoning quality is decisive</i>              | Baja volumen, alta apuesta | Máxima                 |
| <code>balanced</code> | Standard structured generation (specs, critics, seeds)                                         | Media                      | Media-alta             |
| <code>cheap</code>    | Mechanical / high-volume / low-stakes (extraction, normalization)                              | Alta volumen, baja apuesta | Baja                   |
| <code>code</code>     | CODE generation + repair (routes, pages, shell, SQL, fixers) — <i>a coder model when local</i> | Alta, núcleo del producto  | Alta (correctitud)     |

La distinción clave es que `code` **no es** un alias de `premium`. Es un *lane* dedicado. La generación de código es una tarea distinta del razonamiento arquitectónico: cuando se corre local, `premium` quiere un modelo de razonamiento general (Gemma) mientras que `code` quiere un *coder* especializado (Qwen-Coder). El sistema los separa precisamente porque los mejores modelos para una y otra tarea no coinciden — un patrón que el leaderboard confirma (§5).

Cada tier es overridable de forma independiente vía `PUGLIT_MODEL_PREMIUM`, `PUGLIT_MODEL_BALANCED`, `PUGLIT_MODEL_CHEAP`, `PUGLIT_MODEL_CODE`. Y, como se ve en §2, no solo el modelo: el **proveedor** también se puede fijar por tier, de modo que se pueden mezclar nubes y locales en un mismo build.

## Cómo se enrutan los tiers en la práctica

El roster (`web/lib/roster.ts`) define **25 roles × 3 teams = 75 agentes**. Esos roles no eligen un modelo directamente: invocan una de las funciones de alto nivel (`chatText`, `chatJSON`) pasando `MODELS.premium`, `MODELS.balanced`, etc., y el chokepoint `call()` se encarga del resto. La función interna `tierOf(model)` hace el camino inverso — dado un nombre de modelo, recupera a qué tier pertenece — para que el trazado (`run-trace.ts`) y la resolución de proveedor por tier funcionen:

```
function tierOf(model: string): "PREMIUM" | "BALANCED" | "CHEAP" | "CODE" | null {
  if (model === MODELS.code)      return "CODE"
  if (model === MODELS.premium)   return "PREMIUM"
  if (model === MODELS.balanced)  return "BALANCED"
  if (model === MODELS.cheap)     return "CHEAP"
  return null
}
```

Nota un detalle de orden: `code` se chequea **primero**. Si alguien configura `PUGLIT_MODEL_CODE` igual a `PUGLIT_MODEL_PREMIUM` (mismo modelo para ambos), el sistema lo clasifica como `CODE` — el *lane* de código gana el desempate. Esto importa para el ruteo de proveedor por tier y para que el profiler atribuya correctamente la llamada.

---

## 2. Provider Agnosticism

---

Puglit es **provider-agnostic por diseño**. El default open-source corre 100% local en Ollama sin API key y sin costo; y se puede BYO (*bring your own*) cualquier endpoint OpenAI-compatible o Anthropic. Esto no es marketing: está en la tabla `PROVIDERS` y se reduce a una observación arquitectónica — **dos protocolos cubren todo el universo de LLMs**.

### Los cinco providers

| Provider  | Protocolo | Base URL                     | JSON mode | Vision | Necesita key |
|-----------|-----------|------------------------------|-----------|--------|--------------|
| ollama    | openai    | http://localhost:11434/v1    | sí        | no     | no           |
| openai    | openai    | https://api.openai.com/v1    | sí        | sí     | sí           |
| gemini    | openai    | .../v1beta/openai            | sí        | sí     | sí           |
| anthropic | anthropic | https://api.anthropic.com/v1 | no        | sí     | sí           |
| custom    | openai    | http://localhost:11434/v1    | sí        | sí     | no           |

`custom` es la puerta del BYO: apunta a cualquier endpoint OpenAI-compatible (OpenRouter, Together, vLLM, DeepSeek, SiliconFlow, un vLLM propio) seteadando `OPENAI_BASE_URL` / `PUGLIT_BASE_URL`. No es un provider más: es el comodín que absorbe todo el ecosistema de modelos open-weight servidos por terceros.

## Los dos protocolos

El comentario del archivo lo dice sin rodeos: *"Two protocols cover everything"*. El `openai` (OpenAI / Gemini / Ollama / custom — todos comparten la misma *shape* de request/response) y el `anthropic` (la Messages API de Claude, la única distinta, manejada por un adaptador). El chokepoint `call()` despacha según `r.def.protocol`:

```
if (r.def.protocol === "anthropic") out = await callAnthropic(messages, opts, r)
else if (opts.schema && isOllama(r) && allStrings(messages)) out = await callOllamaSchema(...)
else out = await callOpenAICompat(messages, opts, r)
```

El adaptador `callAnthropic` traduce mensajes estilo OpenAI a la Messages API (extrae el `system`, mapea `image_url` → `image` con `source.base64`, clampea `temperature` a 0-1). Para el resto del mundo, `callOpenAICompat` sirve sin tocar nada. Una observación importante de diseño que el propio comentario subraya: los modelos open-weight (Gemma, Hermes, DeepSeek, Qwen, Llama...) **no son providers separados** — son simplemente *nombres de modelo* bajo `ollama` o `custom`. Esto colapsa lo que en otros sistemas serían N integraciones a dos rutas de código.

## Resolución de proveedor: el default cascada y los overrides por tier

Sin nada configurado, un clon open-source arranca **100% local**. El default se resuelve por presencia de credenciales, con un override explícito que siempre gana:

```
const DEFAULT_PROVIDER = (
  process.env.PUGLIT_PROVIDER ||
  (process.env.OPENAI_BASE_URL ? "custom"
    : process.env.OPENAI_API_KEY ? "openai"
    : process.env.ANTHROPIC_API_KEY ? "anthropic"
    : process.env.GEMINI_API_KEY ? "gemini"
    : "ollama")
).toLowerCase()
```

Encima de eso, `resolve(model)` permite fijar provider, base URL y key **por tier**, leyendo `PUGLIT_<TIER>_PROVIDER`, `PUGLIT_<TIER>_BASE_URL`, `PUGLIT_<TIER>_API_KEY`. Esto es lo que habilita el

caso de uso estrella: *premium* → Claude para el blueprint, *cheap* → Gemma local para extracción, todo en el mismo build, decidido por env. El mismo idea que el comentario del header anota: "You can even MIX providers per tier".

---

### 3. Local vs API

---

La infraestructura real es un pod RunPod **A40 48GB**. Esa cifra define la frontera entre lo que corre adentro y lo que sale por API.

#### Lo que entra en el A40

Un A40 con 48GB de VRAM corre cómodo modelos *coder* de **~32B** en cuantización (Q4/Q5), que es exactamente el sweet spot del tier `code`. Los defaults locales en `providerModels("ollama")` son conservadores para que un clon arranque en hardware modesto:

```
default: { premium: "gemma2:27b", balanced: "gemma2", cheap: "gemma2:2b", code: "qwen2.5-coder:72b" }
```

En el pod productivo esos defaults se suben por env a los modelos densos más capaces que la VRAM aguanta: un `gemma2:27b` (o sucesor) para `premium`, un `qwen2.5-coder:32b` / `devstral` / `deepseek-coder` para `code` (los tres equipos genéticos corren *coders* distintos, ver `roster.ts` `TEAMS`: Lean→Qwen, Enterprise→DeepSeek-Coder, Hacker→Devstral). El stock de Ollama en el pod ronda los **~77GB de modelos** descargados. La ventaja del local es total: **sin key, sin costo, sin rate-limit, sin telemetría saliente** — el agente piensa dentro de la caja.

#### Lo que NO entra: los MoE gigantes van por API

Acá está el límite duro. Los dos modelos open-weight *top* de junio 2026 para razonamiento agéntico — **GLM 5.2 Max** y **DeepSeek V4 Pro** — son **Mixture-of-Experts gigantes** en el rango **100B–671B** de parámetros totales. Aunque solo activen una fracción de expertos por token, **el peso completo tiene que residir en VRAM**, y eso desborda con holgura los 48GB del A40. No hay cuantización razonable que meta 671B en un solo pod sin destruir la calidad.

La solución es pragmática y ya soportada por el código: servirlos vía **API OpenAI-compatible** (p. ej. **SiliconFlow**, que hostea ambos) usando el provider `custom`. Se setea `PUGLIT_PREMIUM_PROVIDER=custom`, `PUGLIT_PREMIUM_BASE_URL=<endpoint SiliconFlow>`, `PUGLIT_PREMIUM_API_KEY=...`, `PUGLIT_MODEL_PREMIUM=glm-5.2-max` (o `deepseek-v4-pro`) — y el tier `premium` razona con un MoE de frontera mientras `code` sigue corriendo gratis adentro del A40.

| Clase de modelo                 | Dónde corre       | Ejemplos                                                   | Por qué                                                 |
|---------------------------------|-------------------|------------------------------------------------------------|---------------------------------------------------------|
| Dense ≤32B<br>(coders, general) | Local A40         | qwen2.5-coder:32b, devstral,<br>deepseek-coder, gemma2:27b | Entran en 48GB VRAM<br>cuantizados; sin costo ni key    |
| MoE gigante 100B–<br>671B       | API<br>( custom ) | GLM 5.2 Max, DeepSeek V4 Pro                               | No caben en un pod single; se<br>sirven vía SiliconFlow |
| Frontier cloud                  | API nativa        | Claude Fable 5 / Opus 4.8, GPT 5.5,<br>Gemini 2.5 Pro      | Cerrados; máxima calidad para<br>premium /juez          |

El diseño hace que esta decisión sea **por tier y por env**, no una bifurcación de código. El mismo binario corre full-local en una laptop, full-cloud en un deploy hospedado, o híbrido (coders locales + premium MoE por API) en el pod.

#### 4. Model Selection per Role

El reparto tier→rol sale de cruzar lo que cada tier promete (§1) con lo que cada rol del roster necesita. La regla es: **plata donde decide, barato donde es mecánico, coder donde se escribe código**.

| Rol (familia en roster.ts)                           | Tier     | Justificación                                                                        |
|------------------------------------------------------|----------|--------------------------------------------------------------------------------------|
| Master-spec / contracts / domain architect           | premium  | Diseña el blueprint desde una idea; el razonamiento define toda la build aguas abajo |
| Discovery / researcher / reference-studier           | premium  | Investigación abierta, síntesis; calidad de razonamiento decisiva                    |
| Grand Jury (judge panel, tournament.ts)              | premium  | Discrimina entre 3 blueprints rivales; un juez débil colapsa la presión evolutiva    |
| Adversarial review (3 lentes, adversarial-review.ts) | premium  | Lee el código real, no el claim; debe atrapar lo que un modelo complaciente perdona  |
| SkillOpt optimizer (skill-evolution.ts)              | premium  | Propone ediciones acotadas al skill-doc; entrenamiento del propio swarm              |
| Backend / frontend engineer                          | code     | Genera routes, pages, SQL — el corazón del producto; coder especializado             |
| QA / reliability, swarm-repair (fixers)              | code     | Reparación de phantom-tables, tests; tarea de código                                 |
| Completeness-critic, spec critics, seeds             | balanced | Generación estructurada estándar; ni discovery ni código crudo                       |
| Extracción / normalización (parsers internos)        | cheap    | Mecánico, alto volumen, bajo riesgo                                                  |

La asignación más cargada de consecuencias es **juez y arquitecto en premium**. El sistema de jurado (doc 04) mezcla el veredicto del LLM al 60% con un `objectiveScore` determinístico al 40%, justamente porque un juez subjetivo es ruidoso y complaciente. Pero ese 60% sigue importando: un `premium` flojo arrastra todo el torneo. Por eso el premium es el tier donde más rinde gastar — sea un MoE open-weight por API o un frontier cloud.

El tier `code` es el otro lugar donde la elección es crítica, pero por una razón distinta: **correctitud, no razonamiento**. Acá un *coder* de 32B local suele empatar o ganarle a un generalista cloud más caro en la tarea acotada de emitir un route handler de Next.js que compila. El leaderboard agéntico (§5) y el leaderboard de código no coinciden, y Puglit explota esa brecha.

## 5. The Leaderboard Read

La selección de modelos no es por marca: es por la lectura de dos rankings que **no se correlacionan**. El *Agent Arena* (junio 2026) mide capacidad **agéntica** — uso de herramientas, planificación multi-paso, razonamiento sobre estado — que es lo que importa para `premium` /juez/arquitecto. Los benchmarks de **código** (HumanEval-style, SWE-bench) miden otra cosa: correctitud de generación/reparración, que es lo que importa para `code` .

**Top cloud (frontier, cerrados) — para `premium` cuando hay presupuesto**

| Modelo          | Lectura                                                                   |
|-----------------|---------------------------------------------------------------------------|
| Claude Fable 5  | Tope del Agent Arena; mejor razonamiento agéntico y <i>steerability</i>   |
| Claude Opus 4.8 | Frontier, contexto largo; juez/arquitecto de máxima calidad               |
| GPT 5.5         | Top-3 agéntico; default del provider <code>openai</code> (premium + code) |

**Open self-hosteables / API — para `premium` open-weight y juez**

| Modelo      | Licencia | Posición             | Forma                  | Dónde corre                                   |
|-------------|----------|----------------------|------------------------|-----------------------------------------------|
| GLM 5.2     | MIT      | #10 agéntico         | MoE (Max = gigante)    | densos chicos local; <b>Max vía API</b>       |
| DeepSeek V4 | MIT      | top open agéntico    | MoE (Pro = 671B-class) | <b>Pro vía API</b> (SiliconFlow)              |
| Qwen 3.6    | abierta  | competitivo          | denso/MoE              | densos local; coders → tier <code>code</code> |
| Gemma 4 31B | abierta  | #27 — débil agéntico | denso                  | local (premium económico / vision)            |

La lectura accionable es directa:

- Para `premium` /juez open-weight, el ganador es GLM 5.2 (MIT, #10) o DeepSeek V4 (MIT). Son lo mejor que el mundo abierto ofrece en razonamiento agéntico, y la licencia MIT los hace self-hosteables sin fricción legal. Como sus variantes top son MoE gigantes, entran por API `custom` (§3), no en el pod.
- Gemma 4 31B es fuerte como generalista barato y para vision, pero #27 lo descalifica como juez serio. Sirve de default `premium` económico local (un clon que arranca sin API), no como el cerebro que decide un torneo en producción. Por eso el default local de `premium` es Gemma pero el pod productivo lo sube a GLM/DeepSeek por API.
- Qwen/DeepSeek-Coder/Devstral viven en el tier `code` , local. Acá el leaderboard agéntico es casi irrelevante: lo que manda es el benchmark de código, y un coder 32B local compite de igual a igual con frontier cloud en la tarea acotada de generar/reparrar — gratis y sin rate-limit.

El mensaje de fondo: **agéntico ≠ código**. Puglit no elige "el mejor modelo" sino el mejor modelo *por rol*, y los dos rankings que importan llevan a respuestas distintas.

---

## 6. On Abliterated / Uncensored Models

---

Una tentación recurrente en swarms self-hosteados es usar modelos *abliterated* o *uncensored* — variantes a las que se les removió quirúrgicamente la dirección de rechazo (la *refusal direction*) para que nunca digan "no puedo ayudar con eso". La intuición es: "un agente que genera código no debería negarse a nada".

**Puglit los descarta deliberadamente**, por tres razones técnicas, no ideológicas.

1. **Pierden capacidad.** La abliteración es una intervención destructiva sobre los pesos/activaciones. Quitar la dirección de rechazo no es gratis: degrada de forma medible el razonamiento general y la coherencia en tareas largas. Para un tier `premium` que tiene que *discriminar* entre tres blueprints o *atrapar* un bug que un modelo complaciente perdonaría, perder capacidad es exactamente lo opuesto de lo que se necesita.
2. **Pierden steerability.** El swarm depende de control fino vía prompt y vía parámetros (las RPG stats {creativity, rigor, security, speed, depth} de cada rol se traducen a temperatura/determinismo de Ollama). Un modelo abliterado responde peor a la instrucción de sistema — la misma cirugía que mata el rechazo erosiona la capacidad de seguir *cualquier* dirección fuerte, incluyendo "actúa como auditor adversarial y rechaza lo inseguro". El rol *Adversary* de la revisión de tres lentes (`adversarial-review.ts`) necesita un modelo que **sí** rechace lo abusivo; un uncensored lo sabotea.
3. **El problema que resuelven casi nunca aparece.** En la práctica de generar SaaS full-stack (auth, CRUD, pricing, reviews), los rechazos por contenido son **rarísimos**. Puglit ya maneja la fricción real con su capa de parsing robusto (`parseLoose`, retry estricto) y la salida estructurada gramaticalmente forzada (`callOllamaSchema` con `format`). El costo de un rechazo ocasional es trivial frente al costo permanente de un modelo más tonto y menos dirigible en todos los demás builds.

En síntesis: la abliteración compra una solución a un problema marginal pagando con capacidad y steerability en el camino crítico. Para un sistema cuya ventaja es la calidad del juicio y la disciplina del control, es un mal trade. Puglit usa **modelos base limpios** — Gemma, Qwen, DeepSeek, GLM — y resuelve el formato con decoding constraints, no con lobotomías.

---

## 7. Cost / Quality Tradeoffs

---

El sistema de tiers existe para poner la calidad cara solo donde mueve la aguja. La matriz de decisión, ordenada por dónde rinde el gasto:

| Tier     | Volumen de llamadas | Calidad necesaria  | Dónde correr (recomendado)        | ¿Pagar premium aquí?            |
|----------|---------------------|--------------------|-----------------------------------|---------------------------------|
| premium  | Bajo                | Máxima             | GLM/DeepSeek API o frontier cloud | Sí — decide el torneo           |
| code     | Alto                | Alta (correctitud) | Coder 32B <b>local A40</b>        | No — local empata cloud, gratis |
| balanced | Medio               | Media              | Local denso o <b>cheap</b> -cloud | No                              |
| cheap    | Alto                | Baja               | Modelo chico local (gemma2:2b)    | <b>Nunca</b>                    |

Tres principios que el código encarna:

- **El premium es barato en agregado.** Se invoca pocas veces por build (un blueprint, un panel de jurado, una review), así que poner ahí el modelo más caro impacta poco el costo total y mucho la calidad. Es el mejor ratio de todos.
- **El código es el costo dominante en volumen — por eso es local.** El tier `code` genera decenas de archivos por build. Sacar eso por API sería el grueso de la factura. Correrlo en el A40 lo vuelve **costo marginal cero** sin sacrificar correctitud, porque los coders 32B local son competitivos en la tarea acotada.
- **El profiling cierra el loop.** Cada llamada se traza (`traceCall` en `call()`): tier, prompt size, latencia, ok) y `swarm-profile.ts` (idea *agent-house*) puntúa Cost/Latency/Reliability/Context del build y rankea fixes. El costo no es una estimación de servilleta: es medido por build y atribuido por tier vía `tierOf`.

El worst case a evitar es el inverso de la matriz: premium-cloud para extracción (caro y lento donde no importa) o cheap-local para el juez (barato donde se juega la build entera). El sistema de tiers + overrides por env existe justamente para que esa mala asignación sea explícita y fácil de corregir.

## 8. Recommended Configuration

Tres perfiles, todos sobre el mismo binario, decididos enteramente por variables de entorno.

### Perfil A — Full local (default open-source, \$0)

Nada que configurar. Sin keys, el `DEFAULT_PROVIDER` cae en `ollama` y arranca 100% local. Para subir los defaults conservadores a los modelos del pod A40:

```
PUGLIT_PROVIDER=ollama
PUGLIT_MODEL_PREMIUM=gemma2:27b
PUGLIT_MODEL_BALANCED=gemma2:27b
PUGLIT_MODEL_CHEAP=gemma2:2b
PUGLIT_MODEL_CODE=qwen2.5-coder:32b
# (Enterprise/Hacker teams usan deepseek-coder / devstral vía roster.ts)
```

### Perfil B — Híbrido recomendado para el pod A40 (premium MoE por API, code local)

El sweet spot productivo: razonamiento de frontera open-weight donde decide, coders gratis donde se escribe código.

```
# code / balanced / cheap → local A40 (gratis, sin rate-limit)
PUGLIT_MODEL_CODE=qwen2.5-coder:32b
PUGLIT_MODEL_BALANCED=gemma2:27b
PUGLIT_MODEL_CHEAP=gemma2:2b

# premium → GLM 5.2 Max (MoE gigante) vía SiliconFlow, provider custom
PUGLIT_PREMIUM_PROVIDER=custom
PUGLIT_PREMIUM_BASE_URL=https://api.siliconflow.cn/v1
PUGLIT_PREMIUM_API_KEY=sk-...
PUGLIT_MODEL_PREMIUM=glm-5.2-max           # o: deepseek-v4-pro (ambos MIT)
```

Perfil C — Full frontier cloud (máxima calidad, BYO)

Para un deploy hosteado donde se prioriza calidad sobre costo. Premium en Claude, code en un coder cloud, mezcla por tier:

```
# Premium + juez → Claude (protocolo anthropic, adaptador interno)
PUGLIT_PREMIUM_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-...
PUGLIT_MODEL_PREMIUM=claude-opus-4-8      # o claude-fable-5 (#1 Agent Arena)

# Code → GPT 5.5
PUGLIT_CODE_PROVIDER=openai
OPENAI_API_KEY=sk-...
PUGLIT_MODEL_CODE=gpt-5.5

# Cheap → local Gemma para extracción (mezcla provider por tier)
PUGLIT_CHEAP_PROVIDER=ollama
PUGLIT_MODEL_CHEAP=gemma2:2b
```

Variables de control transversales

Más allá de modelo y provider por tier, el cliente expone flags finos:

| Variable                                      | Efecto                                                                              |
|-----------------------------------------------|-------------------------------------------------------------------------------------|
| PUGLIT_MAX_TOKENS                             | Cap de salida (default 8192; relevante para Anthropic max_tokens )                  |
| PUGLIT_VISION=always never                    | Fuerza/desactiva la ruta de visión (algunos modelos Ollama sí ven)                  |
| PUGLIT_SEND_TEMPERATURE=always never          | Override de wantsTemperature (los modelos gpt-5 / o\ d rechazan temperatura custom) |
| PUGLIT_<TIER>_PROVIDER / _BASE_URL / _API_KEY | Override completo de proveedor por tier                                             |

Verificación: el doctor

El módulo expone `providerInfo()` y `pingProvider()` precisamente para validar la config sin filtrar keys. `providerInfo()` reporta `defaultProvider`, `vision`, `configured` y, por cada tier, el `{ provider, model, baseUrl, protocol, hasKey }` resuelto. `pingProvider()` hace una llamada viva al premium y confirma que responde. Cualquier perfil de arriba se chequea con estas dos funciones antes de lanzar un build — sin adivinar, sin exponer secretos.

## Apéndice — Trazabilidad

| Afirmación                                                     | Fuente verificada                                                                                     |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 4 tiers <code>premium/balanced/cheap/code</code> + semántica   | <code>web/lib/openai.ts</code> <code>MODELS</code> (líneas 54-63)                                     |
| Env overrides <code>PUGLIT_MODEL_*</code>                      | <code>web/lib/openai.ts</code> (líneas 56-62)                                                         |
| 5 providers + 2 protocolos                                     | <code>web/lib/openai.ts</code> <code>PROVIDERS</code> (líneas 22-28)                                  |
| Default provider cascada → ollama                              | <code>web/lib/openai.ts</code> <code>DEFAULT_PROVIDER</code> (líneas 33-40)                           |
| Defaults por provider (openai/gemini/anthropic/ollama)         | <code>web/lib/openai.ts</code> <code>providerModels</code> (líneas 44-51)                             |
| Override de provider/baseURL/key por tier                      | <code>web/lib/openai.ts</code> <code>resolve</code> (líneas 76-89)                                    |
| Adaptador Anthropic (Messages API)                             | <code>web/lib/openai.ts</code> <code>callAnthropic</code> , <code>toAnthropicContent</code> (116-152) |
| Decoding constrained Ollama ( <code>format</code> )            | <code>web/lib/openai.ts</code> <code>callOllamaSchema</code> (163-178)                                |
| <code>tierOf</code> chequea <code>code</code> primero          | <code>web/lib/openai.ts</code> (68-74)                                                                |
| Profiling por llamada (tier/latencia)                          | <code>web/lib/openai.ts</code> <code>call</code> + <code>traceCall</code> (197-211)                   |
| <code>providerInfo</code> / <code>pingProvider</code> (doctor) | <code>web/lib/openai.ts</code> (226-249)                                                              |
| 25 roles × 3 teams = 75 agentes; coders por team               | <code>web/lib/roster.ts</code> <code>TEAMS</code> (vía <code>_SHARED_FACTS.md</code> )                |
| A40 48GB, ~77GB modelos Ollama                                 | <code>_SHARED_FACTS.md</code> (Infrastructure)                                                        |
| Juez blend 60/40, postura adversarial                          | doc 04, <code>web/lib/tournament.ts</code>                                                            |
| Profiler <code>swarm-profile.ts</code> (Cost/Latency/...)      | <code>_SHARED_FACTS.md</code> , <code>web/lib/swarm-profile.ts</code>                                 |

Leaderboard (Agent Arena, jun 2026) y posiciones de modelos (Claude Fable 5 / Opus 4.8 / GPT 5.5 cloud; GLM 5.2 MIT #10, DeepSeek V4 MIT, Qwen 3.6, Gemma 4 31B #27) son datos de contexto de mercado, no del repo. La integración técnica (correrlos vía provider `custom` / `anthropic` por tier) sí está verificada contra el código.